



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

May 15th



Outline

- ◆ Review
- ◆ Array and Pointers
- ◆ Matrix and Pointers

Review

```
#include <stdio.h>
```

```
int main ()
```

```
{ int a=1, b=2;
```

```
int *p1 = &a, *p2 = &b, *pt;
```

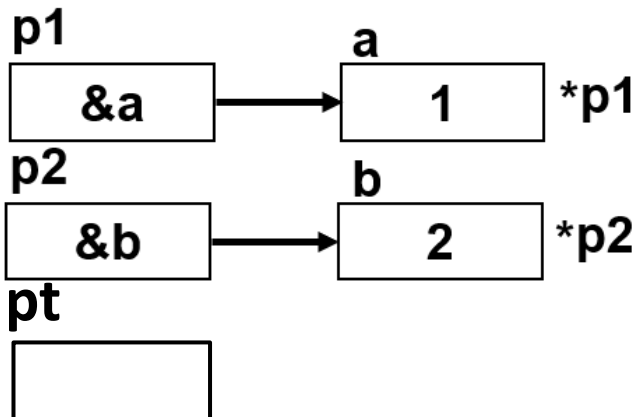
```
printf ("a=%d, b=%d, *p1=%d, *p2=%d\n", a, b, *p1, *p2);
```

```
pt = p1; p1 = p2; p2 = pt;
```

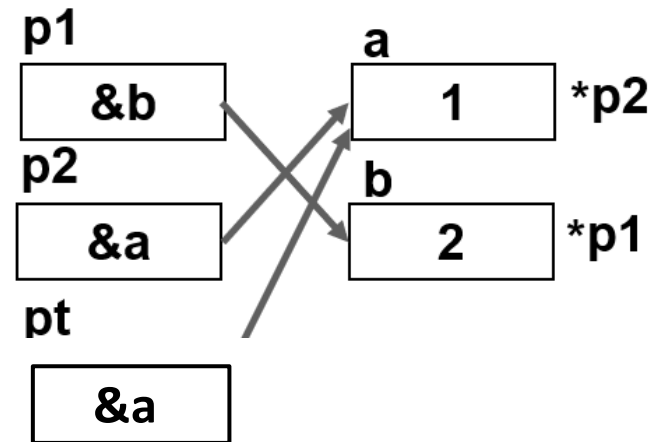
```
printf ("a=%d, b=%d, *p1=%d, *p2=%d\n", a, b, *p1, *p2);
```

```
return 0;
```

```
}
```



```
a = 1; b = 2; *p1 = 1, *p2 = 2  
a = 1; b = 2; *p1 = 2, *p2 = 1
```



Review (cont.)

- `*ip = *ip + 10;`
increments `*ip` by 10.
- `y = *ip + 1;`
takes whatever `ip` points at, adds 1, and assigns the result to `y`
- `*ip += 1`
increments what `ip` points to, as do
`++*ip`
and
`(*ip)++`
The parentheses are necessary in this last example

Exercise 1

Pointer_e2.c

```
#include <stdio.h>

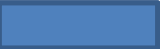
void swap (int *p1,int *p2); //declare

int main(){

    int a,b;

    printf("a = ");
    scanf("%d", &a);
    printf("b = ");
    scanf("%d", &b);

    printf("before swap¥n");
    printf("a = %d, b = %d¥n", a, b);

    swap 
    printf("after swap¥n");
    printf("a = %d, b = %d¥n", a, b);
    return 0;
}
```

```
void swap(int *p1, int*p2) //define
{
    int temp;
    temp=*p1;
    *p1=*p2;
    *p2=temp;
}
```

Exercise 1 (cont.)

Pointer_e2.c

```
#include <stdio.h>

void swap (int *p1,int *p2); //declare

int main(){

    int a,b;

    printf("a = ");
    scanf("%d", &a);
    printf("b = ");
    scanf("%d", &b);

    printf("before swap¥n");
    printf("a = %d, b = %d¥n", a, b);

    swap(&a, &b);
    printf("after swap¥n");
    printf("a = %d, b = %d¥n", a, b);
    return 0;
}
```

```
void swap(int *p1, int*p2) //define
{
    int temp;
    temp=*p1;
    *p1=*p2;
    *p2=temp;
}
```

Outline

- ◆ Review
- ◆ Array and Pointers
- ◆ Matrix and Pointers

Array

The declaration

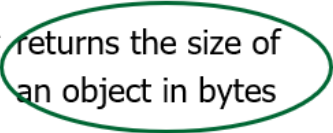
`int a[10];`  **array size in declaration**

defines an array of size 10, that is, a block of 10 consecutive objects named `a[0]`, `a[1]`, ..., `a[9]`.



What is

■ `sizeof(array[3])`?

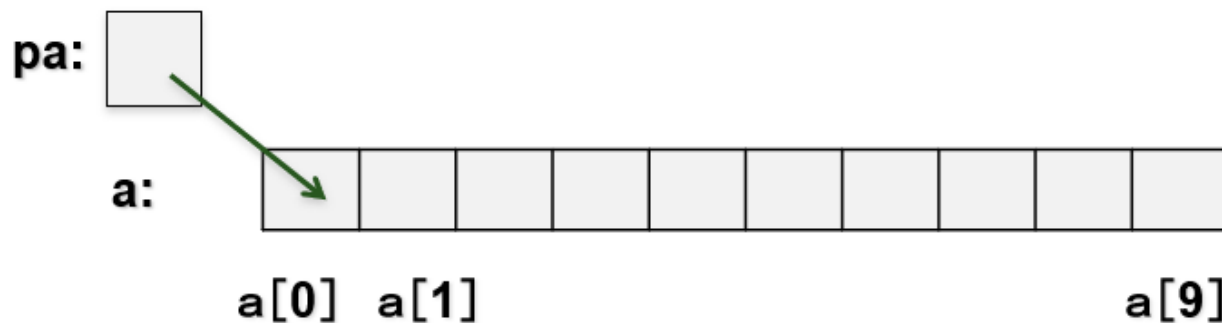
 returns the size of an object in bytes

■ `sizeof(array)`?

Array and Pointers

If `pa` is a pointer to an integer, declared as
`int *pa;`
then the assignment
`pa = &a[0];`

sets `pa` to point to element zero of `a`; that is, `pa` contains the address of `a[0]`.



Array and Pointers (cont.)

```
#include <stdio.h>
```

```
array1.c
```

```
int main(void)
```

```
{
```

```
    int array[5];
```

```
    int i;
```

```
    for (i=0; i<5; i++) array[i] = i + 10;
```

```
    for (i=0; i<5; i++) printf("%d\n", array[i]);
```

```
    for (i=0; i<5; i++) printf("&array[%d] ... %p\n", i, &array[i]);
```

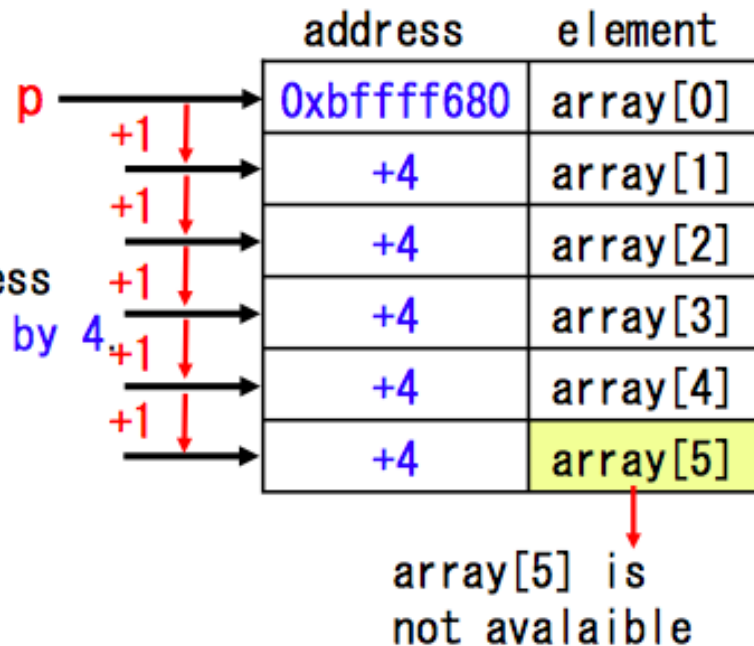
```
    return 0;
```

```
}
```

Array and Pointers (cont.)

use pointers to show the elements of an array.

- because array's type is int, declare pointer to int type :
`int *p;`
- when we increase p by 1, the address which p points will be increased by 4
- At the first step, p points to the first element (array[0]) :
`p = &array[0]`
- increase p until that p points to the last element



Array and Pointers (cont.)

```
#include <stdio.h>
```

array1.c

```
int main(void)
```

```
{
```

```
    int array[5];
```

```
    int i;
```

```
    for (i=0; i<5; i++) array[i] = i + 10;
```

```
    for (i=0; i<5; i++) printf("%d\n", array[i]);
```

```
    for (i=0; i<5; i++) printf("&array[%d] ... %p\n", i, &array[i]);
```

```
    return 0;
```

```
}
```

Try to use pointer in the
for loop condition

Array and Pointers (cont.)

```
#include <stdio.h>
```

array2.c

```
int main(void)
```

```
{
```

```
    int array[5];
```

```
    int *p;
```

```
    int i;
```

```
    for (i = 0; i < 5; i++) array[i] = i + 10;
```

```
    for (p = &array[0]; p != &array[5]; p++) printf("%d\\n", *p);
```

```
    return 0;
```

```
}
```

Array and Pointers (cont.)

the for loop in 「array2.c」 can be revised

```
for (p = &array[0]; p != &array[5]; p++)  
    printf("%d¥n", *p);
```



```
p = &array[0];  
for (i = 0; i < 5; i++) printf("%d¥n, *(p + i));
```

Array and Pointers (cont.)

```
p = &array[0];  
for (i = 0; i < 5; i++) printf("%d\n", *(p + i));
```

`*(p + i)`

by using [] (subscript operator), it can be written

which means,

`p[i] ⇔ *(p + i)`

For example, `p[3] = *(p + 3)`

Array and Pointers (cont.)

- if we declare

```
int array[5];
```

⇒ array becomes the pointer to its first element.

⇒ `array[i] = *(array + i)`

```
p = &array[0];  
for (i = 0; i < 5; i++) printf("%d\n", *(p + i));
```



```
for (i = 0; i < 5; i++) printf("%d\n", *(array + i));
```


Outline

- ◆ Review
- ◆ Array and Pointers
- ◆ Matrix and Pointers

Matrices (Multi-dimensional Arrays)

- If we define

double a[n][n];

➔ a :

a[0][0]	a[0][1]	a[0][2]	...	a[0][n-1]
a[1][0]	a[1][1]	a[1][2]	...	a[1][n-1]
a[2][0]	a[2][1]	a[2][1]	...	a[2][n-1]
			...	
a[n-1][0]	a[n-1][1]	a[n-1][2]	...	a[n-1][n-1]

Matrices (cont.)

```
int a[2][3];
```

		0	1	2
a[0]	0	a[0][0]	a[0][1]	a[0][2]
a[1]	1	a[1][0]	a[1][1]	a[1][2]

a[0],a[1]: rows

	address
a[0]	2000
	2004
	2008
a[1]	2012
	2016
	2020

Matrices (cont.)

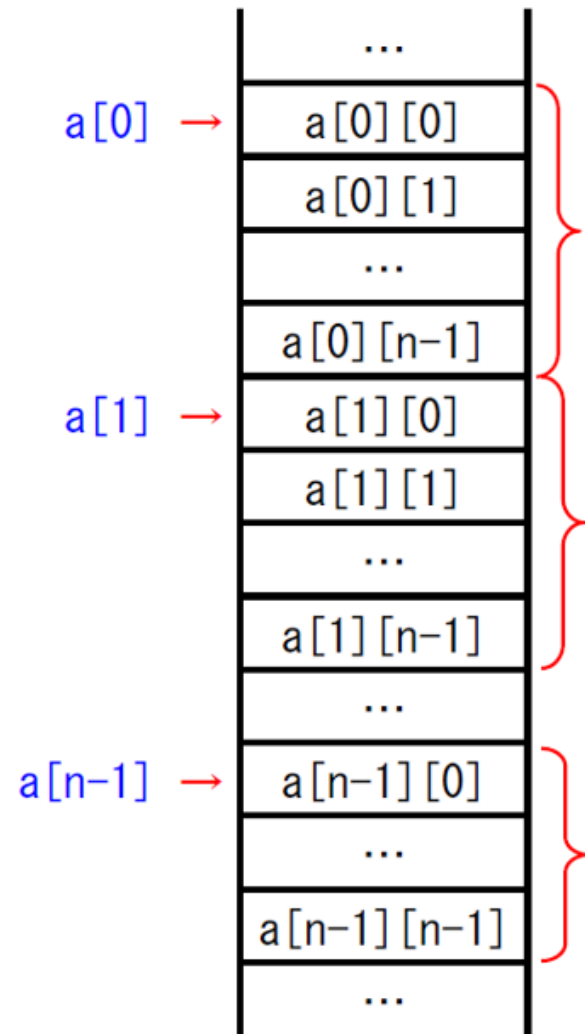
- If we define

`double a[n][n];`

`a[0], a[1], ..., a[n-1]`

- The pointer of 1st element of i-line

`a[i]` ($i=0, 1, \dots, n-1$)



Matrices (cont.)

An array is initialized by:

```
int a[2][3] = { {1, 2, 3}, {4, 5, 6} } ;
```

1	2	3
4	5	6

```
int a[2][3] = {1, 2, 3, 4, 5, 6} ;
```

1	2	3
4	5	6

```
int a[ ][3] = {1, 2, 3, 4, 5, 6} ;
```

The length of row is 2

1	2	3
4	5	6

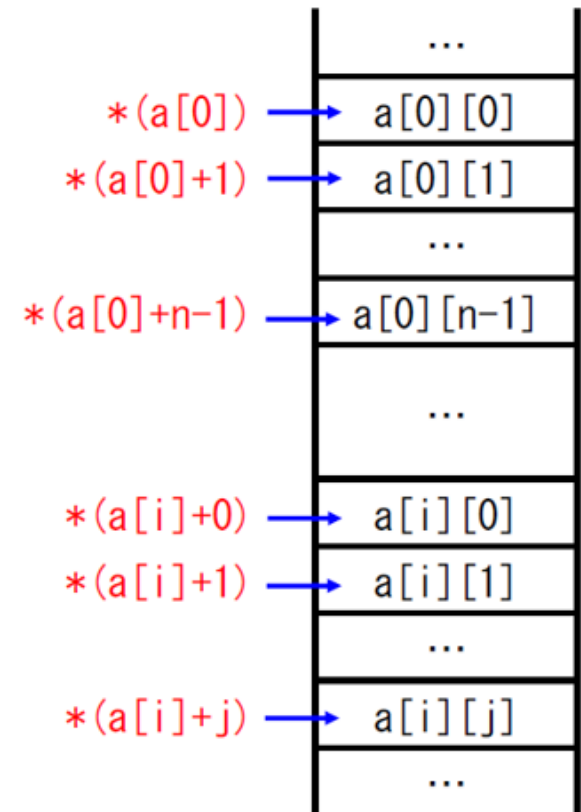
Matrices and Pointers

- The j-element of i-line

$*(a[i] + j)$

- The j-element of i-line

$a[i][j] \Leftrightarrow *(a[i] + j)$



Matrices and Pointers (cont.)

a [2][3]

10	20	30	a+0
11	12	13	a+1



$*(a[1]+2)$

a : pointer to 0th row

a+i : pointer to i-th row

*(a+i): pointer to 0th element of i-th row

*(a+i)+j: pointer to j-th element of i-th row

$*(*(a+1)+2)$

Matrices and Pointers (cont.)

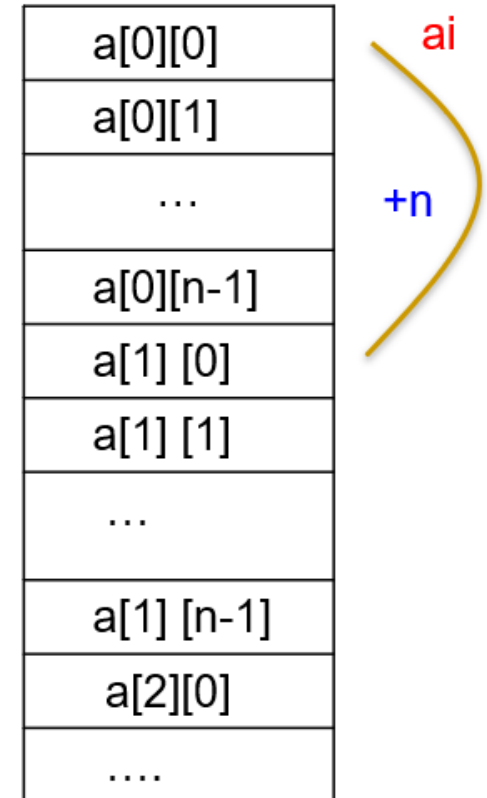
- If we define

```
int *ai = a;
```

$a[0]$: gives the address of 0th array (1D array)

$a[1]$: gives the address of 1th array (1D array)

- ai will be the pointer of the 1st element of the 1st line
- $ai = a = \&a[0][0]$
- $ai += n$
- ai will be the pointer of the 1st element of the next line



Exercise 2

- For

$$A = \begin{pmatrix} 2 & 4 & 6 \\ 3 & 8 & 7 \\ 5 & 7 & 21 \end{pmatrix}, \quad x = \begin{pmatrix} -33 \\ 9 \\ 6 \end{pmatrix}$$

- How to write the program to calculate $(A \cdot x)$ with pointers?

Exercise 2 (cont.)

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int i, j;
```

```
    double x[3] = {-33.0, 9.0, 6.0};
```

```
    double y[3];
```

```
    double a[3][3] = {{2.0, 4.0, 6.0},  
                      {3.0, 8.0, 7.0},  
                      {5.0, 7.0,
```

```
21.0}};
```

```
    double *ai;
```



```
ai= *a;
```

```
for (i=0;i<3;i++){
```

```
    for (j=0;j<3;j++){
```

```
        ai+=3;
```

```
    }  
    printf("y=%n");
```

```
    for (i=0;i<3;i++){  
        printf(" %6.2f", *(y+i));  
    }
```

```
    printf("%n");  
    return 0;}
```

Exercise 2 (cont.)

```
#include <stdio.h>
```

```
int main(void)
{
```

```
    int i, j;
```

```
    double x[3] = {-33.0, 9.0, 6.0};
```

```
    double y[3];
```

```
    double a[3][3] = {{2.0, 4.0, 6.0},
                      {3.0, 8.0, 7.0},
                      {5.0, 7.0,
```

```
21.0}};
```

```
    double *ai;
```



```
ai= *a;
```

```
for (i=0;i<3;i++){
```

```
    *(y+i)=0.0;
```

```
    for (j=0;j<3;j++){
```

```
        *(y+i)+= *(ai+j)*(*x+j));
```

```
    }
```

```
    ai+=3;
```

```
}
```

```
printf("y=%n");
```

```
for (i=0;i<3;i++){
```

```
    printf(" %6.2f", *(y+i));
```

```
}
```

```
printf("%n");
```

```
return 0;}
```